
Adversarial Attack Purification with Diffusion Model on ImageNet

Marco Rayner

10701, Carnegie Mellon University
Carnegie Mellon University
Pittsburgh, PA 15213
mrayner@andrew.cmu.edu

Yuanyu Wang

10701, Carnegie Mellon University
Carnegie Mellon University
Pittsburgh, PA 15213
helenwan@andrew.cmu.edu

Wenhao Xu

10701, Carnegie Mellon University
Carnegie Mellon University
Pittsburgh, PA 15213
wenhaoxu@andrew.cmu.edu

Xinyi Zhang

10701, Carnegie Mellon University
Carnegie Mellon University
Pittsburgh, PA 15213
xinyiz4@andrew.cmu.edu

Abstract

This project investigates methods to degrade and then recover the performance of image classification models. The primary focus is to implement adversarial attacks on image data to reduce the performance of classifiers, and subsequently use a diffusion model for purification to restore performance. For this study, we used the Tiny ImageNet dataset with 200 classes comprised of 100,000 training images and 10,000 test images.

The baseline classifier is EfficientNetB0, fine-tuned on normalized Tiny ImageNet images. Attacks were introduced to the input data to disrupt classification accuracy, with experiments designed to evaluate image attacks. The purification process employed a diffusion model trained to denoise adversarially perturbed images before re-evaluating classification performance.

Results indicate that adversarial attacks significantly reduced classification accuracy, while the diffusion model succeeded in partially recovering performance. Future work includes exploring alternative attack strategies, enhancing the purification process using advanced diffusion model architectures, and scaling the experiments to the full ImageNet dataset or other image data to better assess the robustness and generalization of the approach.

1 Introduction

Image classification is important for many real-world tasks such as disease detection, content identification, and many more automated analysis techniques. Deep learning architectures like the convolutional neural networks are a popular method to perform image classification, and the EfficientNet alteration is version of these which uses compound scaling in order to achieve high performance while maintaining higher efficiency. Real world data is often much more noisy and difficult to deal with than a curated dataset, so developing robustness toward non-standard input or attacked images is also very important.

In this project, we hope to explore the usefulness of diffusion as a method of purification for non-Gaussian noise attacks, which one might intuitively expect to be de-noised with high accuracy. We use three proven attack methods: pixel-swapping, Autoattack and ReColorAdv. All lower our

classifiers accuracy, which we try to recover after running the backward process of the diffusion model.

2 Background and related work

Diffusion models are a recent development for high quality image sampling and generation (Ho, Jain, Abbeel, 2020). They consist of two processes: forward where noise is added in steps, and backward where noise is learned to be removed. This lends them to removing noise from samples, which as shown in Nie et. al (2022) can be used to adversarial purification. There is an entire field exploring different adversarial attacks, which was related to the insertion of noise into the images to the extent that the classifier would end up misclassifying attacked images. Adversarial attacks can manipulate machine learning models to misinterpret input data, leading to incorrect outputs. In the context of adversarial purification, diffusion models have gained attention due to their ability to iteratively denoise inputs. This iterative process aligns well with the requirements for adversarial defense, as it enables robust removal of complex noise patterns introduced by adversarial attacks. Existing work has explored various ways to enhance purification performance, including tuning the diffusion process and optimizing denoising steps for specific types of adversarial noise. The integration of diffusion models into adversarial defense opens new possibilities for addressing the ongoing challenge of adversarial robustness.

3 Data

We used Tiny ImageNet dataset, which is a smaller version of ImageNet that contains 200 classes. Each class has 500 training images and 50 validation images. The images are resized to 64x64 pixels. To use them in our classifier model, we resized them to 3x224x224. And the construction of our attacked datasets was obtained within the same shape as input image samples. The dataset we considered as the most suitable candidate because the ImageNet Dataset is one of the most popular and widely-used datasets in computer vision, and demonstrates sufficient features and potentials in multiple downstream tasks for the visual representation learning. Here is the Google Drive Folder link for our data storage including all attack dataset: https://drive.google.com/drive/folders/1ZGYA8h2cJmTxiJLYM89Zo796oEtWiljj?usp=drive_link.

4 Methods

The system architecture, as shown in the Figure 1, We first fine-tune a pre-trained classifier, then we generate adversarial images by swapping pixels of the target image randomly with another image, until the classifier classifies the target image as another class. We diffuse the adversarial image slightly by adding random noise to it via the forward process and recover the clean image via the generative backward process. Then the generated purified images would be passed into the fine-tuned classifier again for measuring the accuracy improvement of the attack erasure.

4.1 Classifier Fine-Tuning

The baseline for our model is the accuracy of the classifier model on adversarial images before they are processed by the diffusion model for purification. For the classifier, we leverage the pre-trained EfficientNet-B0 from the Hugging Face open-source platform with the fine-tuning of converting the last output layer to 200 classes on the Tiny ImageNet. Train/validation/test datasets are 100,000, 5,000 and 5,000 respectively. Trained for 10 epochs with learning rate as 1e-4 and weight decay as 1e-5, the best classifier model is stored at the highest validation accuracy checkpoint (early stopping). This achieved a baseline test accuracy of 79%, as shown in Figure 2. Overfitting is observed beyond 5 epochs, actions like hyperparameter tuning might be carried out later for better performance.

4.2 Adversarial Attacks

We then implemented our three attack methods: pixel-swap, AutoAttack and ReColorAdv. Pixel swapping was done by taking the first image of the next class and randomly swapping in a pixel

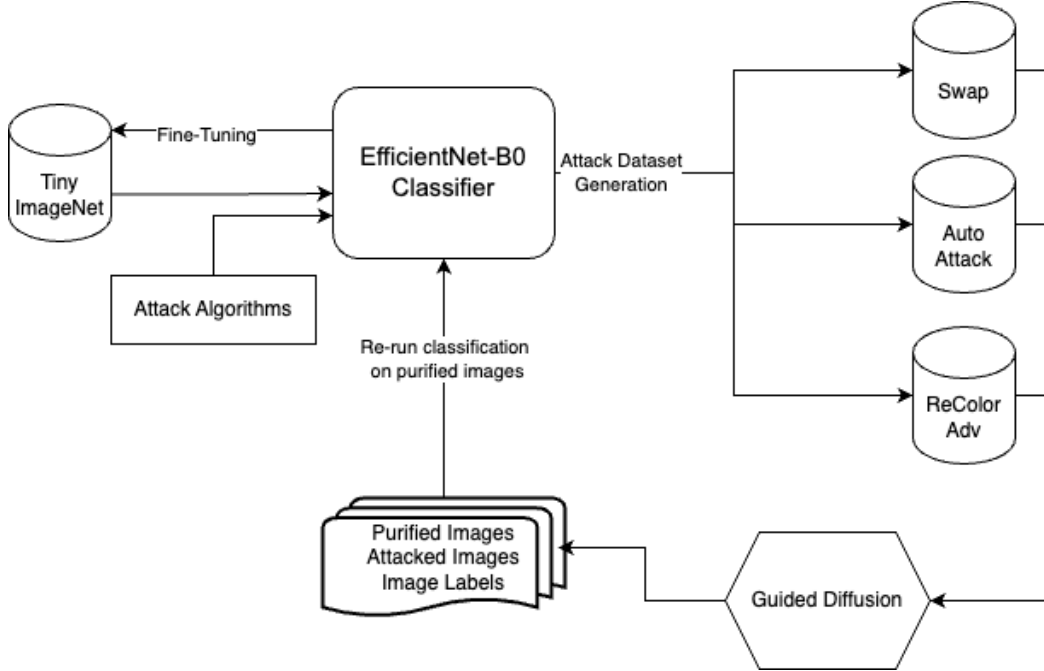


Figure 1: Visualization of our pipeline.

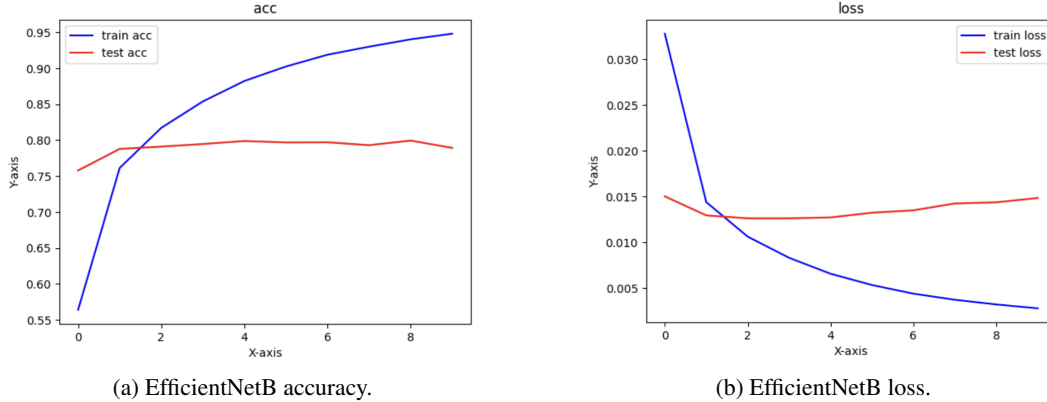


Figure 2: Classifier Fine-tuning model loss and accuracy

from that image across all color channels. There are 50176 total pixels to sample from and this was done in three degrees: 125, 1254, and 12544 steps. The effects on our classifier can be found in Table 1. AutoAttack(Croce & Hein, 2020) runs an ensemble of two untargeted attack methods APGD-CE and APGD-DLR with L-infinity norm and Epsilon = 0.03. We use L-infinity norm to measure the maximum pixel difference between two images and use 0.03 as the maximum allowed perturbation value. Both APGD-CE and APGD-DLR methods aim to maximize their loss functions to make the model predict incorrectly. APGD-CE uses Cross-Entropy as its loss function and APGD-DLR uses Difference of Logits Ratio as its loss function. Both methods compute gradients from their loss functions, then apply different random transformations on gradients. We update the images using this transformed averaged gradient, and finally, the changes are projected within the L-infinity norm bound of 0.03. For ensemble, if an image is successfully created by APGD-CE we use that result, otherwise we try APGD-DLR to generate an adversarial image. According to Figure 4, the adversarial image looks very similar to the original image, but it successfully fooled the model to making incorrect predictions. Since diffusion models are sensitive to image colors, we use ReColorAdv(Laidlaw & Feizi, 2019) to test if the model is robust against adversarial attacks that

Table 1: Accuracy and loss of classifier by steps

Steps	Accuracy	Loss
0 (Baseline)	79.57%	0.0289
125	78.65%	0.0300
1254	62.11%	0.0581
12544	0.75%	0.2439

modify image colors. ReColorAdv changes image colors to create adversarial images by using a color transformation function instead of pixel perturbations. ReColorAdv uses a loss function combining CW loss and perturbation loss by weighted sum. Perturbation loss controls the magnitude of changes. ReColorAdv transforms images into CIELUV color space and uses Adam optimizer with learning rate 0.001, running for 50-100 iterations to find optimal color transformations. We also set the maximum color change as 0.06. According to Figure 5, Color transformation provides more natural images than other methods. The motivation for trying all these methods is that intuitively, simply adding Gaussian noise as an attack is too suited to a diffusion model, as it learns to remove noise from a certain distribution at each step. However, the more complicated attacks allow us to identify specifically where the diffusion model is good at "defending" these attacks.

4.3 Diffusion Model

For the implementation of the diffusion model, we came up with two possible approaches. First approach is to download a pre-trained diffusion model from hugging face. However, these diffusion models are mainly set for image generation, using text and input and output an image. In order to make the diffusion model do noise removal, we'll need to make the pre-trained diffusion model take the image as input, while also changing its inner steps. Instead of starting with an image of pure noise and denoise it step by step in typical image generation tasks, we need the diffusion model to add noise gradually to a given image and then denoise it to remove the attack from the given adversarial image.

For our second approach, we decide to reuse the code of "Diffusion Models for Adversarial Purification" [4], the paper we take inspiration from. However their code base DiffPure [5] is very complicated. Instead of having only the diffusion model, their code is set up to take an image dataset, a classifier, an adversarial attack method and a diffusion model. When the code gets executed, it will read the image dataset, apply the adversarial attack on the given classifier to generate adversarial images, and then use the diffusion model to clean the adversarial images and feed the cleaned images back into the classifier. Thus the diffusion model is not isolated, but wrapped inside other functionalities. What's worse, the author's code requires CUDA. After some digging in the code, we found that we cannot run the code with CPU because some custom layers are optimized in performance by applying some custom CUDA settings, which require CUDA to be available else the code will crash.

After discovering that the code base cannot be run on CPU with some minor changes, we look at ways to run the code somewhere with CUDA. One possible solution is to run the code in a VM that has GPU access. However, since AWS credit is not available, we decided to use colab, which provides a simple way to get access to GPU. One challenge of running the codebase on colab is environment setup. While the author's code uses docker to manage dependencies, docker does not work well on colab. Therefore, we manually looked at the docker's environment file and installed the necessary packages with pip installs.

Since we already have our adversarial image dataset generated and stored as pickle files, we comment out all the code involving the classifier and attacks. The classifiers we use are not in one of the classifier options the authors implemented. Additionally, it is too much work to try to incorporate our classifier into the framework of the author's code. Therefore, we decided to change the forward function of the author's diffusion model to make it return the cleaned image, instead of having it pass the cleaned image to the classifier and return the predicted label. We store the output of the diffusion model as a pickle file and evaluate it separately.

One other major thing we change is data loading. The original code expects datasets in LMDB format. We reused the data loading pipeline of the authors by locating the data loading code and

altered it to read pickle files instead of LMDB dataset. Although our classifier takes image of size (3,224,224) while the diffusion model expects image of size (3,256,256). The author’s code automatically resize the input image tensors, feeding (3,256,256) images to the diffusion model, then resize it back to (3,224,224) to be re-evaluated by the classifier. We stored the purified dataset as pickle files. We finally fed this purified dataset back to our classifier.

5 Results

The experiment setup includes procedures of data loading, pre-trained guided diffusion model parameters loading, hyperparameter tuning and output storage for further classification. We deployed the Stochastic Differential Equations for score-based diffusion modeling (SDE Diffusion) with model configuration including number of channels as 256, number of heads as 4 with 64 channels each, together with the linear noise scheduler and learned σ within the diffusion process.

Due to the shortage of the compute resources, the majority of our batch processing was carried out on the Colab Notebook with GPU A100 compute engine. We obtained 150 purified samples for each of three attacked datasets with batch size as 5 for 30 iterations. Re-classification for the performance of purification was conducted on 450 samples.

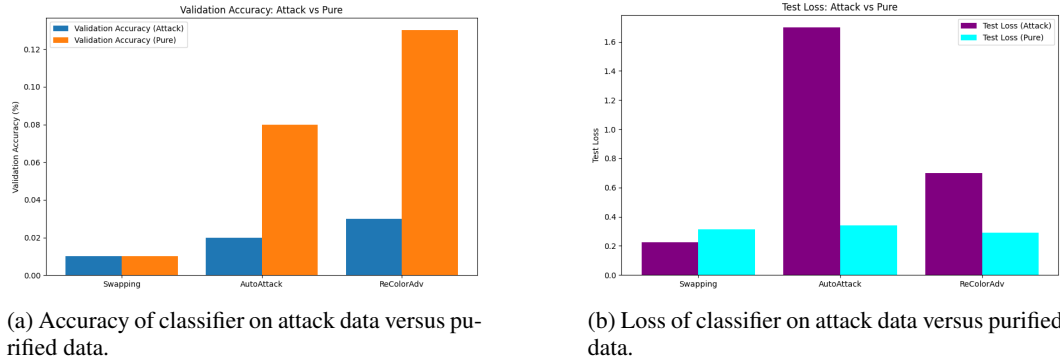


Figure 3: Purification Result

As seen in Figure 3, the model’s classification accuracy is destroyed by all methods of attack. Swapping pixels gives it around $\sim 1\%$ accuracy, auto-attack gives it $\sim 2\%$ accuracy, and ReColorAdv gives it $\sim 3\%$ accuracy. After the diffusion purification is performed the model’s accuracy then becomes $\sim 1\%$ for swapping pixels, $\sim 8\%$ for auto-attack, and $\sim 13\%$ for ReColorAdv. This shows definite improvements on the purified data for recovering some of the classifiers performance. The reason pixel swapping does not show an improvement may be because the information lost is too great so the image is so destroyed that the diffusion process does not help restore the image. As seen in Figure 6, some nuance of the image does not seem to be kept after the purification. In Figure 7 and 8, we see that to the human eye, auto-attack and ReColorAdv are less destructive attacks for fooling our classifier, so the purification seems to maintain most of the structure of the image. Interestingly, and it may have to do with the normalization of our images and how the diffusion model was trained, but all diffusion results seem to include deeper colors with higher contrast.

6 Discussion and Analysis

Diffusion models are sensitive to color variations because the models rely on color information during their denoising steps, so we use ReColorAdv to test if the model is robust against adversarial attacks that modify image colors. However, the results are not good enough. In the future, we may explore other models to test their robustness against color-based adversarial attacks. Despite not recovering the high performance of the classifier on the original data, the results we found are conclusive. While we did not have the resources to run our pipeline enough times to create confidence intervals, the changes in accuracy are high enough that it is difficult to attribute them to chance. The results also show that diffusion as a purification method seems more suited to certain kinds of attacks than others.

The extent of this project was limited by the course not being able to approve AWS credits in time for our project, so we had to drastically reduce the scope of our analysis (see Piazza @582) to what we were able to run on Google Colab. This meant that our accuracy results and inference steps had to be performed on a sub-sample of the validation data. Future work could include extending this project to perform more involved attacks and extending the purification and analysis to the entire dataset.

References

- [1] Croce, Francesco and Hein, Matthias. “Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks”. In: (2020).
- [2] Ho, Jonathan, Jain, Ajay, and Abbeel, Pieter. “Denoising Diffusion Probabilistic Models”. In: *CoRR* abs/2006.11239 (2020). arXiv: 2006.11239. URL: <https://arxiv.org/abs/2006.11239>.
- [3] Laidlaw, Cassidy and Feizi, Soheil. “Functional Adversarial Attacks”. In: *CoRR* abs/1906.00001 (2019). arXiv: 1906.00001. URL: <http://arxiv.org/abs/1906.00001>.
- [4] Nie, Weili et al. *Diffusion Models for Adversarial Purification*. 2022. arXiv: 2205.07460 [cs.LG]. URL: <https://arxiv.org/abs/2205.07460>.
- [5] NVlabs. *DiffPure: A GitHub repository for diffusion purification methods*. <https://github.com/NVlabs/DiffPure>. Accessed: 2024-12-10. 2024.
- [6] Tan, Mingxing and Le, Quoc V. “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks”. In: *CoRR* abs/1905.11946 (2019). arXiv: 1905.11946. URL: <http://arxiv.org/abs/1905.11946>.

7 Appendix

7.1 Figures



Figure 4: Comparison of the original image (left) and the adversarial image (right) generated using AutoAttack(rand). The adversarial image is misclassified as label 121 by the EfficientNetB0 model

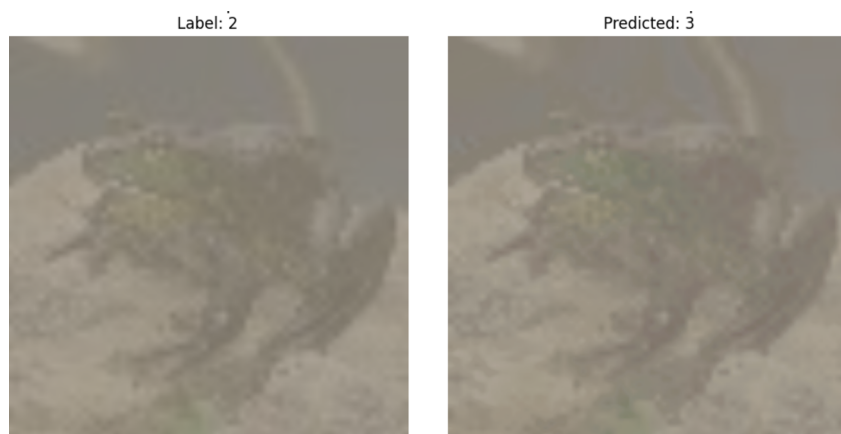


Figure 5: Comparison of the original image (left) and the adversarial image (right) generated using ReColorAdv. The adversarial image is misclassified as label 3 by the EfficientNetB0 model

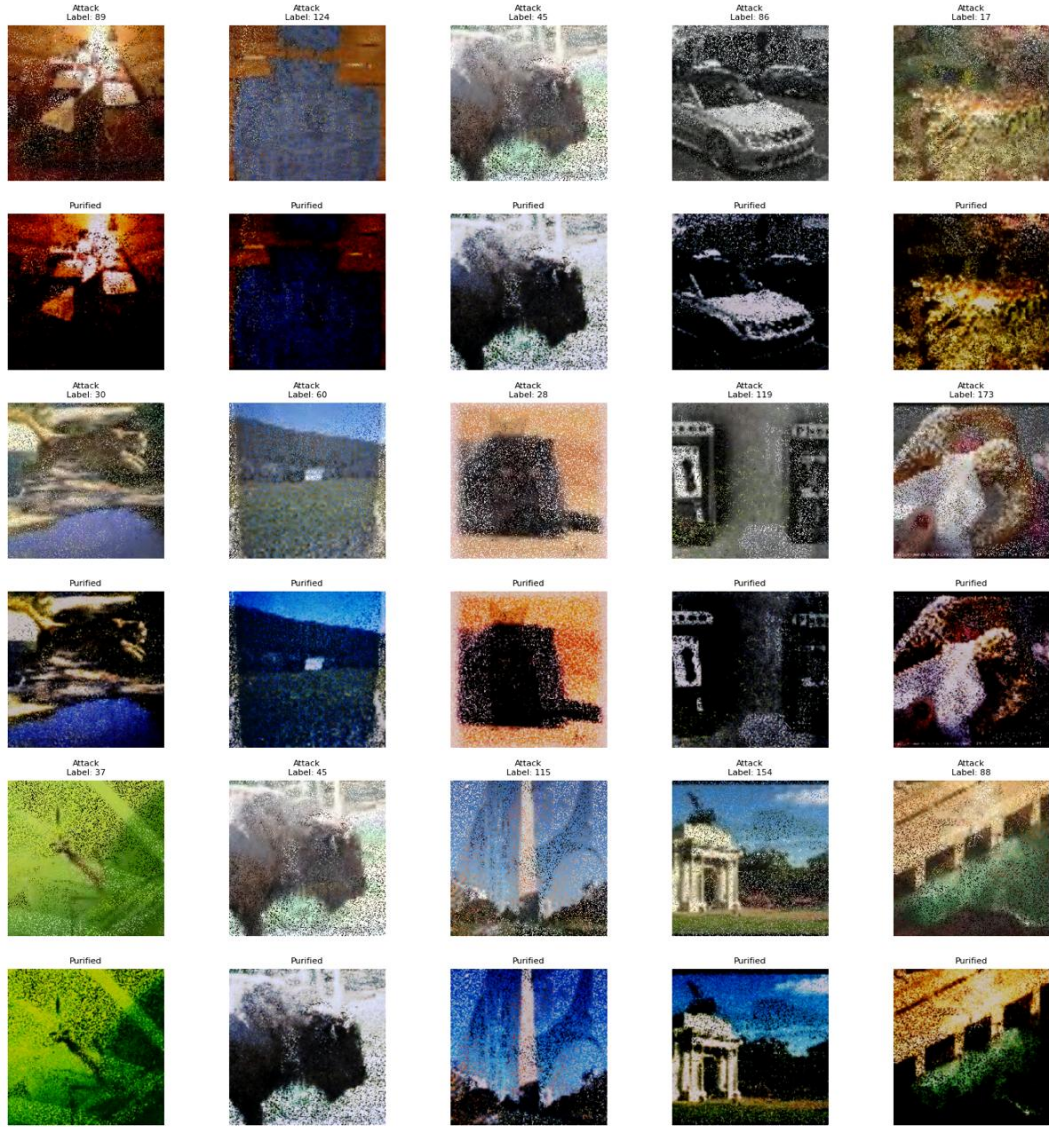


Figure 6: Purification of Swapping.

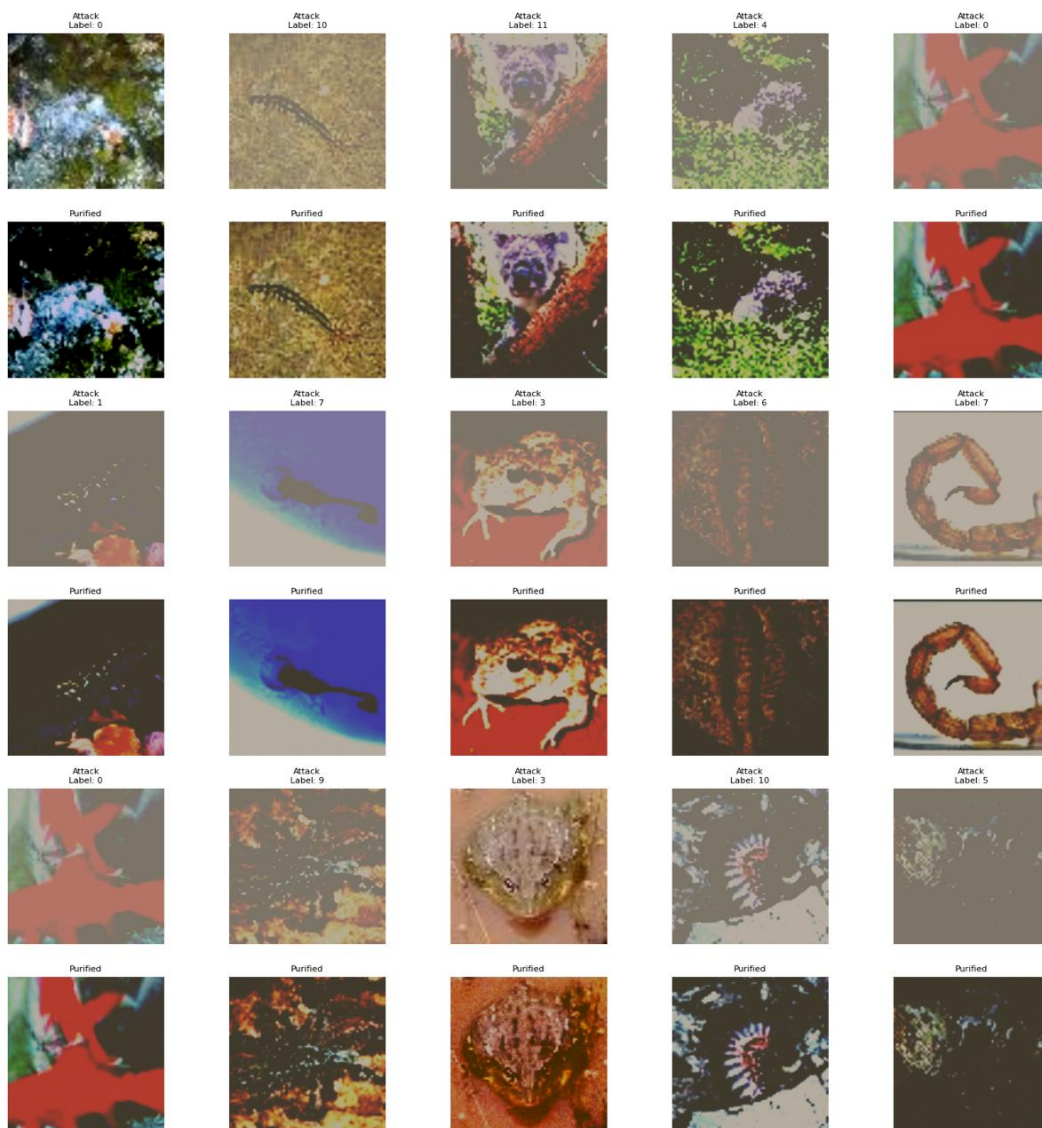


Figure 7: Purification of Auto-Attack.

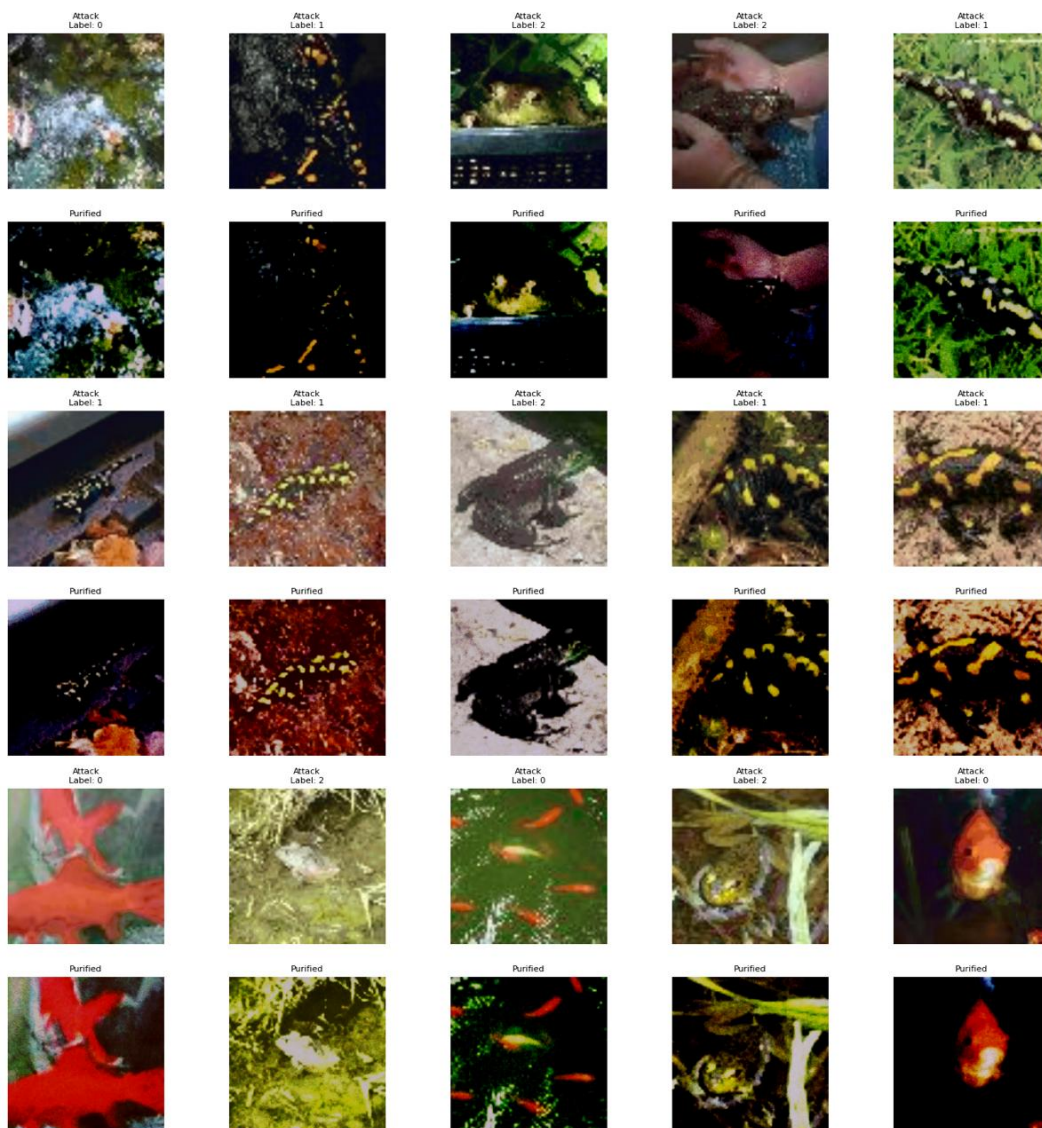


Figure 8: Purification of ReColorAdv.